Database Management Systems

Summary: Week 5 - 8

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

January 26, 2022

Week 5 Summary

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies
- **Anomaly** : inconsistencies that can arise due to data changes in a database with insertion, deletion, and update

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies
- **Anomaly** : inconsistencies that can arise due to data changes in a database with insertion, deletion, and update
  - These problems occur in poorly planned, un-normalised databases where all the data is stored in one table (a flat-file database)

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies
- **Anomaly** : inconsistencies that can arise due to data changes in a database with insertion, deletion, and update
  - These problems occur in poorly planned, un-normalised databases where all the data is stored in one table (a flat-file database)
- There can be three kinds of anomalies:

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies
- **Anomaly** : inconsistencies that can arise due to data changes in a database with insertion, deletion, and update
  - These problems occur in poorly planned, un-normalised databases where all the data is stored in one table (a flat-file database)
- There can be three kinds of anomalies:
  - Insertions Anomaly

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies
- **Anomaly** : inconsistencies that can arise due to data changes in a database with insertion, deletion, and update
  - These problems occur in poorly planned, un-normalised databases where all the data is stored in one table (a flat-file database)
- There can be three kinds of anomalies:
  - Insertions Anomaly
  - Deletion Anomaly

Module 21 Recap

- **Redundancy** : having multiple copies of same data in the database.
  - This problem arises when a database is not normalized
  - It leads to anomalies
- **Anomaly** : inconsistencies that can arise due to data changes in a database with insertion, deletion, and update
  - These problems occur in poorly planned, un-normalised databases where all the data is stored in one table (a flat-file database)
- There can be three kinds of anomalies:
  - Insertions Anomaly
  - Deletion Anomaly
  - Update Anomaly

Redundancy and Anomaly

- **Insertions Anomaly**

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget
- **Deletion Anomaly**

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget
- **Deletion Anomaly**
  - When deletion of a data record results in losing some unrelated information that was stored as part of the record that was deleted from a table

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget
- **Deletion Anomaly**
  - When deletion of a data record results in losing some unrelated information that was stored as part of the record that was deleted from a table
  - We delete the last Instructor of a Department from `instructor_with_department`, we lose building and budget information

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget
- **Deletion Anomaly**
  - When deletion of a data record results in losing some unrelated information that was stored as part of the record that was deleted from a table
  - We delete the last Instructor of a Department from `instructor_with_department`, we lose building and budget information
- **Update Anomaly**

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget
- **Deletion Anomaly**
  - When deletion of a data record results in losing some unrelated information that was stored as part of the record that was deleted from a table
  - We delete the last Instructor of a Department from `instructor_with_department`, we lose building and budget information
- **Update Anomaly**
  - When a data is changed, which could involve many records having to be changed, leading to the possibility of some changes being made incorrectly

Redundancy and Anomaly

- **Insertions Anomaly**
  - When the insertion of a data record is not possible without adding some additional unrelated data to the record
  - We cannot add an Instructor in `instructor_with_department` if the department does not have a building or budget
- **Deletion Anomaly**
  - When deletion of a data record results in losing some unrelated information that was stored as part of the record that was deleted from a table
  - We delete the last Instructor of a Department from `instructor_with_department`, we lose building and budget information
- **Update Anomaly**
  - When a data is changed, which could involve many records having to be changed, leading to the possibility of some changes being made incorrectly
  - When the budget changes for a Department having large number of Instructors in `instructor_with_department` application may miss some of them

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **First Normal Form (1NF)** if

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **First Normal Form (1NF)** if
  - the domains of all attributes of R are atomic

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **First Normal Form (1NF)** if
  - the domains of all attributes of R are atomic
  - the value of each attribute contains only a single value from that domain

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **First Normal Form (1NF)** if
  - the domains of all attributes of R are atomic
  - the value of each attribute contains only a single value from that domain
- Non-atomic values complicate storage and encourage redundant (repeated) storage of data

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **First Normal Form (1NF)** if
  - the domains of all attributes of R are atomic
  - the value of each attribute contains only a single value from that domain
- Non-atomic values complicate storage and encourage redundant (repeated) storage of data
  - Example: Set of accounts stored with each customer, and set of owners stored with each account

First Normal Form (1NF)

- A domain is **atomic** if its elements are considered to be indivisible units
- Examples of non-atomic domains:
  - Set of names, composite attributes
  - Identification numbers like CS101 that can be broken up into parts
- A relational schema R is in **First Normal Form (1NF)** if
  - the domains of all attributes of R are atomic
  - the value of each attribute contains only a single value from that domain
- Non-atomic values complicate storage and encourage redundant (repeated) storage of data
  - Example: Set of accounts stored with each customer, and set of owners stored with each account
- We assume all relations are in first normal form

Functional Dependencies

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$

Functional Dependencies

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The **functional dependency (FD)**

$$\alpha \rightarrow \beta$$

holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

Functional Dependencies

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The **functional dependency (FD)**

$$\alpha \rightarrow \beta$$

  holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A, B)$ with a specific instance.

Functional Dependencies

- Let R be a relation schema $\alpha \subseteq R$ and $\beta \subseteq R$
- The **functional dependency (FD)**

$$\alpha \rightarrow \beta$$

  holds on R if and only if for any legal relations $r(R)$, whenever any two tuples t_1 and t_2 of r agree on the attributes α, they also agree on the attributes β. That is,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- Example: Consider $r(A, B)$ with a specific instance.
- On this instance, $A \rightarrow B$ does NOT hold, but $B \rightarrow A$ does hold. So we cannot have tuples like (2, 4), or (3, 5), or (4, 7) added to the current instance.

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F
- A new FD obtained by applying the axioms is said to the logically implied by F

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F
- A new FD obtained by applying the axioms is said to the logically implied by F
- The process of generations of FDs terminate after finite number of steps and we call it the **Closure Set** F^+ for FDs F. This is the set of all FDs logically implied by F

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F
- A new FD obtained by applying the axioms is said to the logically implied by F
- The process of generations of FDs terminate after finite number of steps and we call it the **Closure Set** F^+ for FDs F. This is the set of all FDs logically implied by F
- Clearly, $F \subseteq F^+$

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F
- A new FD obtained by applying the axioms is said to the logically implied by F
- The process of generations of FDs terminate after finite number of steps and we call it the **Closure Set** F^+ for FDs F. This is the set of all FDs logically implied by F
- Clearly, $F \subseteq F^+$
- These axioms are

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F
- A new FD obtained by applying the axioms is said to the logically implied by F
- The process of generations of FDs terminate after finite number of steps and we call it the **Closure Set** F^+ for FDs F. This is the set of all FDs logically implied by F
- Clearly, $F \subseteq F^+$
- These axioms are
  - **Sound** (generate only functional dependencies that actually hold), and

Functional Dependencies : Armstrong's Axioms

- Given a set of Functional Dependencies F, we can infer new dependencies by the **Armstrong's Axioms**:
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These axioms can be repeatedly applied to generate new FDs and added to F
- A new FD obtained by applying the axioms is said to the logically implied by F
- The process of generations of FDs terminate after finite number of steps and we call it the **Closure Set** F^+ for FDs F. This is the set of all FDs logically implied by F
- Clearly, $F \subseteq F^+$
- These axioms are
  - **Sound** (generate only functional dependencies that actually hold), and
  - **Complete** (eventually generate all functional dependencies that hold)

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds
  - **Decomposition**: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds
  - **Decomposition**: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - **Pseudotransitivity**: if $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds
  - **Decomposition**: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - **Pseudotransitivity**: if $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds
- The above rules can be inferred from basic Armstrong's axioms (and hence are not included in the basic set). They can be proven independently too

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds
  - **Decomposition**: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - **Pseudotransitivity**: if $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds
- The above rules can be inferred from basic Armstrong's axioms (and hence are not included in the basic set). They can be proven independently too
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds
  - **Decomposition**: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - **Pseudotransitivity**: if $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds
- The above rules can be inferred from basic Armstrong's axioms (and hence are not included in the basic set). They can be proven independently too
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$

Functional Dependencies : Armstrong's Axioms: Derived Rules

- **Additional Derived Rules:**
  - **Union**: if $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds
  - **Decomposition**: if $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds
  - **Pseudotransitivity**: if $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds
- The above rules can be inferred from basic Armstrong's axioms (and hence are not included in the basic set). They can be proven independently too
  - **Reflexivity**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
  - **Augmentation**: if $\alpha \rightarrow \beta$, then $\gamma\alpha \rightarrow \gamma\beta$
  - **Transitivity**: if $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
  - 1. Is AG a super key?

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
  - 1. Is AG a super key?
  - 1a. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$ (Yes)

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
  - 1. Is AG a super key?
  - 1a. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$ (Yes)
  - 2. Is any subset of AG a superkey?

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
  - 1. Is AG a super key?
  - 1a. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$ (Yes)
  - 2. Is any subset of AG a superkey?
  - 2a. Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$

Functional Dependencies : Closure of Attribute Sets: Example

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$
- $(AG)^+$
  - 1 result $= AG$
  - 2 result $= ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
  - 3 result $= ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
  - 4 result $= ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- Is AG a candidate key?
  - 1. Is AG a super key?
  - 1a. Does $AG \rightarrow R$? == Is $(AG)^+ \supseteq R$ (Yes)
  - 2. Is any subset of AG a superkey?
  - 2a. Does $A \rightarrow R$? == Is $(A)^+ \supseteq R$
  - 2b. Does $G \rightarrow R$? == Is $(G)^+ \supseteq R$

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R
- Example schema not in BCNF:

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R
- Example schema not in BCNF:
  - `instr_dept(ID, name, salary, dept_name, building, budget)`

BCNF: Boyce-Codd Normal Form

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R
- Example schema not in BCNF:
  - `instr_dept(ID, name, salary, dept_name, building, budget)`
  - because the non-trivial dependency `dept_name` $\rightarrow$ `building, budget` holds on `instr_dept`, but `dept_name` is not a superkey

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`
  - `dept_name` $\rightarrow$ `building, budget`

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`
  - `dept_name` $\rightarrow$ `building, budget`
- `inst_dept` is replaced by

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`
  - `dept_name` $\rightarrow$ `building, budget`
- `inst_dept` is replaced by
  - $(\alpha \cup \beta) =$ (`dept_name`, `building`, `budget`)

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`
  - `dept_name` $\rightarrow$ `building, budget`
- `inst_dept` is replaced by
  - $(\alpha \cup \beta) =$ (`dept_name`, `building`, `budget`)
  - `dept_name` $\rightarrow$ `building, budget`

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`
  - `dept_name` $\rightarrow$ `building, budget`
- `inst_dept` is replaced by
  - $(\alpha \cup \beta) =$ (`dept_name`, `building`, `budget`)
  - `dept_name` $\rightarrow$ `building, budget`
  - $(R - (\beta - \alpha)) =$ (`ID`, `name`, `salary`, `dept_name`)

BCNF (2): Decomposition

- If in schema R and a non-trivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, we decompose R into:
  - $\alpha \cup \beta$
  - $(R - (\beta - \alpha))$
- In our example,
  - $\alpha =$ `dept_name`
  - $\beta =$ `building, budget`
  - `dept_name` $\rightarrow$ `building, budget`
- `inst_dept` is replaced by
  - $(\alpha \cup \beta) =$ (`dept_name`, `building`, `budget`)
  - `dept_name` $\rightarrow$ `building, budget`
  - $(R - (\beta - \alpha)) =$ (`ID`, `name`, `salary`, `dept_name`)
  - `ID` $\rightarrow$ `name, salary, dept_name`

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$
- at least one of the following holds:

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$
- at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$
- at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$
- at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R
  - Each attribute A in $\beta - \alpha$ is contained in a candidate key for R (Note: Each attribute may be in a different candidate key)

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$
- at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R
  - Each attribute A in $\beta - \alpha$ is contained in a candidate key for R (Note: Each attribute may be in a different candidate key)
- If a relation is in BCNF it is in 3NF (since in BCNF one of the first two conditions above must hold)

3NF: Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:
- $\alpha \rightarrow \beta \in F^+$
- at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R
  - Each attribute A in $\beta - \alpha$ is contained in a candidate key for R (Note: Each attribute may be in a different candidate key)
- If a relation is in BCNF it is in 3NF (since in BCNF one of the first two conditions above must hold)
- Third condition is a minimal relaxation of BCNF to ensure dependency preservation (will see why later)

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$
  - $A^+ = AC$ in $\{A \rightarrow C, AB \rightarrow C\}$

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$
  - $A^+ = AC$ in $\{A \rightarrow C, AB \rightarrow C\}$
- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$
  - $A^+ = AC$ in $\{A \rightarrow C, AB \rightarrow C\}$
- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
  - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C

Extraneous Attributes

- Consider a set F of FDs and the FD $\alpha \rightarrow \beta$ in F.
- Attribute A is extraneous in α if $A \in \alpha$ and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$ and the set of FDs $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.
- **Note**: Implication in the opposite direction is trivial in each of the cases above, since a 'stronger' functional dependency always implies a weaker one
- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
  - B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$
  - $A^+ = AC$ in $\{A \rightarrow C, AB \rightarrow C\}$
- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
  - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C
  - $AB^+ = ABCD$ in $\{A \rightarrow C, AB \rightarrow D\}$

Equivalence of Sets of Functional Dependencies

- Let F & G are two functional dependency sets.

Equivalence of Sets of Functional Dependencies

- Let F & G are two functional dependency sets.
- These two sets F & G are equivalent if $F^+ = G^+$. That is:
  $(F^+ = G^+) \Leftrightarrow (F^+ \Rightarrow G \text{ and } G^+ \Rightarrow F)$

Equivalence of Sets of Functional Dependencies

- Let F & G are two functional dependency sets.
- These two sets F & G are equivalent if $F^+ = G^+$. That is:
  $(F^+ = G^+) \Leftrightarrow (F^+ \Rightarrow G \text{ and } G^+ \Rightarrow F)$
- Equivalence means that every functional dependency in F can be inferred from G, and every functional dependency in G an be inferred from F

Equivalence of Sets of Functional Dependencies

- Let F & G are two functional dependency sets.
- These two sets F & G are equivalent if $F^+ = G^+$. That is:
  $(F^+ = G^+) \Leftrightarrow (F^+ \Rightarrow G \text{ and } G^+ \Rightarrow F)$
- Equivalence means that every functional dependency in F can be inferred from G, and every functional dependency in G an be inferred from F
- F and G are equal only if

Equivalence of Sets of Functional Dependencies

- Let F & G are two functional dependency sets.
- These two sets F & G are equivalent if $F^+ = G^+$. That is: $(F^+ = G^+) \Leftrightarrow (F^+ \Rightarrow G \text{ and } G^+ \Rightarrow F)$
- Equivalence means that every functional dependency in F can be inferred from G, and every functional dependency in G an be inferred from F
- F and G are equal only if
  - F **covers** G: Means that all functional dependency of G are logically numbers of functional dependency set $F \Rightarrow F^+ \supseteq G$.

Equivalence of Sets of Functional Dependencies

- Let F & G are two functional dependency sets.
- These two sets F & G are equivalent if $F^+ = G^+$. That is:
  $(F^+ = G^+) \Leftrightarrow (F^+ \Rightarrow G \text{ and } G^+ \Rightarrow F)$
- Equivalence means that every functional dependency in F can be inferred from G, and every functional dependency in G an be inferred from F
- F and G are equal only if
  - F **covers** G: Means that all functional dependency of G are logically numbers of functional dependency set $F \Rightarrow F^+ \supseteq G$.
  - G **covers** F: Means that all functional dependency of F are logically members of functional dependency set $G \Rightarrow G^+ \supseteq F$.

Equivalence of Sets of Functional Dependencies (2)

Condition	**CASES**	**CASES**	**CASES**	**CASES**
F Covers G	True	True	False	False
G Covers F	True	False	True	False
Result	F=G		G-F (Unclear)	No Comparison

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$
- Intuitively, a Canonical cover of F is a minimal set of FDs

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$
- Intuitively, a Canonical cover of F is a minimal set of FDs
  - Equivalent to F

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$
- Intuitively, a Canonical cover of F is a minimal set of FDs
  - Equivalent to F
  - Having no redundant FDs

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$
- Intuitively, a Canonical cover of F is a minimal set of FDs
  - Equivalent to F
  - Having no redundant FDs
  - No redundant parts of FDs

Canonical Cover

- A **Canonical Cover** for F is a set of dependencies F_c such that ALL the following properties are satisfied:
  - $F^+ = F_c^+$ Or,
  - F logically implies all dependencies in F_c
  - F_c logically implies all dependencies in F
- No functional dependency in F_c contains an extraneous attribute
- Each left side of functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$
- Intuitively, a Canonical cover of F is a minimal set of FDs
  - Equivalent to F
  - Having no redundant FDs
  - No redundant parts of FDs
  - Minimal / Irreducible Set of Functional Dependencies

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- For example: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- For example: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow CD \Rightarrow A \rightarrow C$ and $A \rightarrow D$ (2) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- For example: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow CD \Rightarrow A \rightarrow C$ and $A \rightarrow D$ (2) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$
  - In the reverse: (1) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$ (2) $A \rightarrow C$, $A \rightarrow D \Rightarrow A \rightarrow CD$

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- For example: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow CD \Rightarrow A \rightarrow C$ and $A \rightarrow D$ (2) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$
  - In the reverse: (1) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$ (2) $A \rightarrow C$, $A \rightarrow D \Rightarrow A \rightarrow CD$
- For example: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- For example: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow CD \Rightarrow A \rightarrow C$ and $A \rightarrow D$ (2) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$
  - In the reverse: (1) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$ (2) $A \rightarrow C$, $A \rightarrow D \Rightarrow A \rightarrow CD$
- For example: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C \Rightarrow A \rightarrow AC$ (2) $A \rightarrow AC$, $AC \rightarrow D \Rightarrow A \rightarrow D$

Canonical Cover : Example

- For example: $A \rightarrow C$ is redundant in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$
- Parts of a functional dependency may be redundant
- For example: on RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow CD \Rightarrow A \rightarrow C$ and $A \rightarrow D$ (2) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$
  - In the reverse: (1) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C$ (2) $A \rightarrow C$, $A \rightarrow D \Rightarrow A \rightarrow CD$
- For example: on LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
  - In the forward: (1) $A \rightarrow B$, $B \rightarrow C \Rightarrow A \rightarrow C \Rightarrow A \rightarrow AC$ (2) $A \rightarrow AC$, $AC \rightarrow D \Rightarrow A \rightarrow D$
  - In the reverse: $A \rightarrow D \Rightarrow AC \rightarrow D$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies
- To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies
- To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:
  - $R_1 \cup R_2 = R$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies
- To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:
  - $R_1 \cup R_2 = R$
  - $R_1 \cap R_2 \neq \emptyset$

Lossless Join Decomposition

- For the case of $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- A decomposition of R into R_1 and R_2 is **lossless join** if at least one of the following dependencies is in F^+:
  - $R_1 \cap R_2 \rightarrow R_1$
  - $R_1 \cap R_2 \rightarrow R_2$
- The above functional dependencies are a sufficient condition for lossless join decomposition; the dependencies are a necessary condition only if all constraints are functional dependencies
- To Identify whether a decomposition is lossy or lossless, it must satisfy the following conditions:
  - $R_1 \cup R_2 = R$
  - $R_1 \cap R_2 \neq \emptyset$
  - $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$

Lossless Join Decomposition : Example

- $R = (A, B, C)$

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B)$, $R_2 = (A, C)$

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B)$, $R_2 = (A, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B)$, $R_2 = (A, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
  - Not dependency preserving

Lossless Join Decomposition : Example

- $R = (A, B, C)$
- $F = \{A \rightarrow B, B \rightarrow C\}$
- Can be decomposed in two different ways
- $R_1 = (A, B)$, $R_2 = (B, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
  - Dependency preserving
- $R_1 = (A, B)$, $R_2 = (A, C)$
  - Lossless-join decomposition: $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
  - Not dependency preserving
  - (cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

- If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

- If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive
- Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

- If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive
- Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if
  - $F_1 \cup F_2 \equiv F$ { Decomposition Preserving Dependency }

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

- If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive
- Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if
  - $F_1 \cup F_2 \equiv F$ { Decomposition Preserving Dependency }
  - If $F_1 \cup F_2 \subset F$ { Decomposition NOT Preserving Dependency } and

Dependency Preservation

- Let F_i be the set of dependencies F^+ that include only attributes in R_i
- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \cdots \cup F_n)^+ = F^+$$

- If it is not, then checking updates for violation of functional dependencies may require computing joins, which is expensive
- Let R be the original relational schema having FD set F. Let R_1 and R_2 having FD set F_1 and F_2 respectively, are the decomposed sub-relations of R. The decomposition of R is said to be preserving if
  - $F_1 \cup F_2 \equiv F$ { Decomposition Preserving Dependency }
  - If $F_1 \cup F_2 \subset F$ { Decomposition NOT Preserving Dependency } and
  - $F_1 \cup F_2 \supset F$ { this is not possible }

Dependency Preservation : Example

- $R(A, B, C, D)$

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$
- $F_2 = \{B \rightarrow BC, C \rightarrow CB\}$

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$
- $F_2 = \{B \rightarrow BC, C \rightarrow CB\}$
- $F_3 = \{C \rightarrow CD, D \rightarrow DC\}$

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$
- $F_2 = \{B \rightarrow BC, C \rightarrow CB\}$
- $F_3 = \{C \rightarrow CD, D \rightarrow DC\}$
- $F' = F_1 \cup F_2 \cup F_3$.

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$
- $F_2 = \{B \rightarrow BC, C \rightarrow CB\}$
- $F_3 = \{C \rightarrow CD, D \rightarrow DC\}$
- $F' = F_1 \cup F_2 \cup F_3$.
- Checking for: $D \rightarrow A$ in F'^{+}

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$
- $F_2 = \{B \rightarrow BC, C \rightarrow CB\}$
- $F_3 = \{C \rightarrow CD, D \rightarrow DC\}$
- $F' = F_1 \cup F_2 \cup F_3$.
- Checking for: $D \rightarrow A$ in F'^{+}
- $D \rightarrow C$ (from R_3), $C \rightarrow B$ (from R_2), $B \rightarrow A$ (from R_1) : $D \rightarrow A$ (By Transitivity)

Dependency Preservation : Example

- $R(A, B, C, D)$
- $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, D \rightarrow A\}$
- Decomposition: $R_1(A, B)$, $R_2(B, C)$, $R_3(C, D)$
- $A \rightarrow B$ is preserved on table R_1
- $B \rightarrow C$ is preserved on table R_2
- $C \rightarrow D$ is preserved on table R_3
- We have to check whether the one remaining FD: $D \rightarrow A$ is preserved or not.
- $F_1 = \{A \rightarrow AB, B \rightarrow BA\}$
- $F_2 = \{B \rightarrow BC, C \rightarrow CB\}$
- $F_3 = \{C \rightarrow CD, D \rightarrow DC\}$
- $F' = F_1 \cup F_2 \cup F_3$.
- Checking for: $D \rightarrow A$ in F'^{+}
- $D \rightarrow C$ (from R_3), $C \rightarrow B$ (from R_2), $B \rightarrow A$ (from R_1) : $D \rightarrow A$ (By Transitivity)
- Hence **all dependencies are preserved**.

Week 6 Summary

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly
- Most common technique for the Schema Refinement is **decomposition**.

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly
- Most common technique for the Schema Refinement is **decomposition**.
- **Goal of Normalization**: Eliminate Redundancy

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly
- Most common technique for the Schema Refinement is **decomposition**.
- **Goal of Normalization**: Eliminate Redundancy
  - Redundancy refers to repetition of same data or duplicate copies of same data stored in different locations

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly
- Most common technique for the Schema Refinement is **decomposition**.
- **Goal of Normalization**: Eliminate Redundancy
  - Redundancy refers to repetition of same data or duplicate copies of same data stored in different locations
- Normalization is used for mainly two purpose:

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly
- Most common technique for the Schema Refinement is **decomposition**.
- **Goal of Normalization**: Eliminate Redundancy
  - Redundancy refers to repetition of same data or duplicate copies of same data stored in different locations
- Normalization is used for mainly two purpose:
  - Eliminating redundant (useless) data

Normalization or Schema Refinement

- **Normalization** or Schema Refinement is a technique of organizing the data in the database
- A systematic approach of decomposing tables to eliminate data redundancy and undesirable characteristics
  - Insertion Anomaly
  - Update Anomaly
  - Deletion Anomaly
- Most common technique for the Schema Refinement is **decomposition**.
- **Goal of Normalization**: Eliminate Redundancy
  - Redundancy refers to repetition of same data or duplicate copies of same data stored in different locations
- Normalization is used for mainly two purpose:
  - Eliminating redundant (useless) data
  - Ensuring data dependencies make sense, that is, data is logically stored

Normalization and Normal Forms

- A normal form specifies a set of conditions that the relational schema must satisfy in terms of its constraints - they offer varied levels of guarantee for the design

Normalization and Normal Forms

- A normal form specifies a set of conditions that the relational schema must satisfy in terms of its constraints - they offer varied levels of guarantee for the design
- Normalization rules are divided into various normal forms. Most common normal forms are:

Normalization and Normal Forms

- A normal form specifies a set of conditions that the relational schema must satisfy in terms of its constraints - they offer varied levels of guarantee for the design
- Normalization rules are divided into various normal forms. Most common normal forms are:
  - **First Normal Form (1NF)**

Normalization and Normal Forms

- A normal form specifies a set of conditions that the relational schema must satisfy in terms of its constraints - they offer varied levels of guarantee for the design
- Normalization rules are divided into various normal forms. Most common normal forms are:
  - **First Normal Form (1NF)**
  - **Second Normal Form (2NF)**

Normalization and Normal Forms

- A normal form specifies a set of conditions that the relational schema must satisfy in terms of its constraints - they offer varied levels of guarantee for the design
- Normalization rules are divided into various normal forms. Most common normal forms are:
  - **First Normal Form (1NF)**
  - **Second Normal Form (2NF)**
  - **Third Normal Form (3NF)**

Normalization and Normal Forms

- A normal form specifies a set of conditions that the relational schema must satisfy in terms of its constraints - they offer varied levels of guarantee for the design
- Normalization rules are divided into various normal forms. Most common normal forms are:
  - **First Normal Form (1NF)**
  - **Second Normal Form (2NF)**
  - **Third Normal Form (3NF)**
- Informally, a relational database relation is often described as 'normalized' if it meets third normal form. Most 3NF relations are free of insertion, update, and deletion anomalies

1NF: First Normal Form

- A relation is in **First Normal Form** if and only if all underlying domains contain atomic values only (doesn't have multivalued attributes (MVA))

1NF: First Normal Form

- A relation is in **First Normal Form** if and only if all underlying domains contain atomic values only (doesn't have multivalued attributes (MVA))
- `STUDENT(Sid, Sname, Cname)`

1NF: First Normal Form

- A relation is in **First Normal Form** if and only if all underlying domains contain atomic values only (doesn't have multivalued attributes (MVA))
- `STUDENT(Sid, Sname, Cname)`

Students		
SID	**Sname**	**Cname**
S1	A	C,C++
S2	B	C++, DB
S3	A	DB
SID : Primary Key		

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:
- Let R be a relational Schema and X, Y, A be the attribute sets over R where X: Any Candidate Key, Y: Proper Subset of Candidate Key, and A: Non Prime Attribute

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:
- Let R be a relational Schema and X, Y, A be the attribute sets over R where X: Any Candidate Key, Y: Proper Subset of Candidate Key, and A: Non Prime Attribute
- If $Y \rightarrow A$ exists in R, then R is not in 2NF.

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:
- Let R be a relational Schema and X, Y, A be the attribute sets over R where X: Any Candidate Key, Y: Proper Subset of Candidate Key, and A: Non Prime Attribute
- If $Y \rightarrow A$ exists in R, then R is not in 2NF.
- $(Y \rightarrow A)$ is a Partial dependency only if

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:
- Let R be a relational Schema and X, Y, A be the attribute sets over R where X: Any Candidate Key, Y: Proper Subset of Candidate Key, and A: Non Prime Attribute
- If $Y \rightarrow A$ exists in R, then R is not in 2NF.
- $(Y \rightarrow A)$ is a Partial dependency only if
  - Y: Proper subset of Candidate Key

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:
- Let R be a relational Schema and X, Y, A be the attribute sets over R where X: Any Candidate Key, Y: Proper Subset of Candidate Key, and A: Non Prime Attribute
- If $Y \rightarrow A$ exists in R, then R is not in 2NF.
- $(Y \rightarrow A)$ is a Partial dependency only if
  - Y: Proper subset of Candidate Key
  - A: Non Prime Attribute

2NF: Second Normal Form

- Relation R is in **Second Normal Form (2NF)** only iff :
  - R is in 1NF and
  - R contains **no Partial Dependency**
- **Partial Dependency**:
- Let R be a relational Schema and X, Y, A be the attribute sets over R where X: Any Candidate Key, Y: Proper Subset of Candidate Key, and A: Non Prime Attribute
- If $Y \rightarrow A$ exists in R, then R is not in 2NF.
- $(Y \rightarrow A)$ is a Partial dependency only if
  - Y: Proper subset of Candidate Key
  - A: Non Prime Attribute
- A **prime attribute** of a relation is an attribute that is a part of a candidate key of the relation

3NF: Third Normal Form

- Let R be the relational schema.

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or
  - Every element of $A - X$, the set difference between A and X, is a prime attribute (i.e., each attribute in $A - X$ is contained in some candidate key)

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or
  - Every element of $A - X$, the set difference between A and X, is a prime attribute (i.e., each attribute in $A - X$ is contained in some candidate key)
- **[Simple Statement]** A relational schema R is in 3NF if for every FD $X \rightarrow A$ associated with R either

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or
  - Every element of $A - X$, the set difference between A and X, is a prime attribute (i.e., each attribute in $A - X$ is contained in some candidate key)
- **[Simple Statement]** A relational schema R is in 3NF if for every FD $X \rightarrow A$ associated with R either
  - $A \subseteq X$ (that is, the FD is trivial) or

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or
  - Every element of $A - X$, the set difference between A and X, is a prime attribute (i.e., each attribute in $A - X$ is contained in some candidate key)
- **[Simple Statement]** A relational schema R is in 3NF if for every FD $X \rightarrow A$ associated with R either
  - $A \subseteq X$ (that is, the FD is trivial) or
  - X is a superkey of R or

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or
  - Every element of $A - X$, the set difference between A and X, is a prime attribute (i.e., each attribute in $A - X$ is contained in some candidate key)
- **[Simple Statement]** A relational schema R is in 3NF if for every FD $X \rightarrow A$ associated with R either
  - $A \subseteq X$ (that is, the FD is trivial) or
  - X is a superkey of R or
  - A is part of some candidate key (not just superkey!)

3NF: Third Normal Form

- Let R be the relational schema.
- **[E. F. Codd,1971]** R is in 3NF only if:
  - R should be in 2NF
  - R should not contain transitive dependencies (OR, Every non-prime attribute of R is non-transitively dependent on every key of R)
- **[Carlo Zaniolo, 1982]** Alternately, R is in 3NF iff for each of its functional dependencies $X \rightarrow A$, at least one of the following conditions holds:
  - X contains A (that is, A is a subset of X, meaning $X \rightarrow A$ is trivial functional dependency), or
  - X is a superkey, or
  - Every element of $A - X$, the set difference between A and X, is a prime attribute (i.e., each attribute in $A - X$ is contained in some candidate key)
- **[Simple Statement]** A relational schema R is in 3NF if for every FD $X \rightarrow A$ associated with R either
  - $A \subseteq X$ (that is, the FD is trivial) or
  - X is a superkey of R or
  - A is part of some candidate key (not just superkey!)
- A relation in 3NF is naturally in 2NF

3NF Decomposition: Motivation

- There are some situations where

3NF Decomposition: Motivation

- There are some situations where
  - BCNF is not dependency preserving, and

3NF Decomposition: Motivation

- There are some situations where
  - BCNF is not dependency preserving, and
  - Efficient checking for FD violation on updates is important

3NF Decomposition: Motivation

- There are some situations where
  - BCNF is not dependency preserving, and
  - Efficient checking for FD violation on updates is important
- **Solution**: define a weaker normal form, called **Third Normal Form (3NF)**

3NF Decomposition: Motivation

- There are some situations where
  - BCNF is not dependency preserving, and
  - Efficient checking for FD violation on updates is important
- **Solution**: define a weaker normal form, called **Third Normal Form (3NF)**
  - Allows some redundancy (with resultant problems; as seen above)

3NF Decomposition: Motivation

- There are some situations where
  - BCNF is not dependency preserving, and
  - Efficient checking for FD violation on updates is important
- **Solution**: define a weaker normal form, called **Third Normal Form (3NF)**
  - Allows some redundancy (with resultant problems; as seen above)
  - But functional dependencies can be checked on individual relations without computing a join

3NF Decomposition: Motivation

- There are some situations where
  - BCNF is not dependency preserving, and
  - Efficient checking for FD violation on updates is important
- **Solution**: define a weaker normal form, called **Third Normal Form (3NF)**
  - Allows some redundancy (with resultant problems; as seen above)
  - But functional dependencies can be checked on individual relations without computing a join
  - There is always a lossless-join, dependency-preserving decomposition into 3NF

3NF Decomposition : Testing for 3NF

- **Optimization**: Need to check only FDs in F, need not check all FDs in F^+.

3NF Decomposition : Testing for 3NF

- **Optimization**: Need to check only FDs in F, need not check all FDs in F^+.
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.

3NF Decomposition : Testing for 3NF

- **Optimization**: Need to check only FDs in F, need not check all FDs in F^+.
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R

3NF Decomposition : Testing for 3NF

- **Optimization**: Need to check only FDs in F, need not check all FDs in F^+.
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R
- This test is rather more expensive, since it involve finding candidate keys

3NF Decomposition : Testing for 3NF

- **Optimization**: Need to check only FDs in F, need not check all FDs in F^+.
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R
- This test is rather more expensive, since it involve finding candidate keys
- Testing for 3NF has been shown to be NP-hard

3NF Decomposition : Testing for 3NF

- **Optimization**: Need to check only FDs in F, need not check all FDs in F^+.
- Use attribute closure to check for each dependency $\alpha \rightarrow \beta$, if α is a superkey.
- If α is not a superkey, we have to verify if each attribute in β is contained in a candidate key of R
- This test is rather more expensive, since it involve finding candidate keys
- Testing for 3NF has been shown to be NP-hard
- Decomposition into 3NF can be done in polynomial time

3NF Decomposition : Algorithm

- **Given**: relation R, set F of functional dependencies

3NF Decomposition : Algorithm

- **Given**: relation R, set F of functional dependencies
- **Find**: decomposition of R into a set of 3NF relation R_i

3NF Decomposition : Algorithm

- **Given**: relation R, set F of functional dependencies
- **Find**: decomposition of R into a set of 3NF relation R_i
- **Algorithm**:

3NF Decomposition : Algorithm

- **Given**: relation R, set F of functional dependencies
- **Find**: decomposition of R into a set of 3NF relation R_i
- **Algorithm**:
  - 1. Eliminate redundant FDs, resulting in a canonical cover F_c of F

3NF Decomposition : Algorithm

- **Given**: relation R, set F of functional dependencies
- **Find**: decomposition of R into a set of 3NF relation R_i
- **Algorithm**:
  - 1. Eliminate redundant FDs, resulting in a canonical cover F_c of F
  - 2. Create a relation $R_i = XY$ for each FD $X \rightarrow Y$ in F_c

3NF Decomposition : Algorithm

- **Given**: relation R, set F of functional dependencies
- **Find**: decomposition of R into a set of 3NF relation R_i
- **Algorithm**:
  - 1. Eliminate redundant FDs, resulting in a canonical cover F_c of F
  - 2. Create a relation $R_i = XY$ for each FD $X \rightarrow Y$ in F_c
  - 3. If the key K of R does not occur in any relation R_i, create one more relation $R_i = K$

3NF Decomposition : Example

- Relation schema:

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`
- We first compute a canonical cover

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`
- We first compute a canonical cover
  - `branch_name` is extraneous in the RHS of the 1st dependency

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`
- We first compute a canonical cover
  - `branch_name` is extraneous in the RHS of the 1st dependency
  - No other attribute is extraneous, so we get $F_c =$

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`
- We first compute a canonical cover
  - `branch_name` is extraneous in the RHS of the 1st dependency
  - No other attribute is extraneous, so we get $F_c =$
    - `customer_id, employee_id` $\rightarrow$ `type`

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`
- We first compute a canonical cover
  - `branch_name` is extraneous in the RHS of the 1st dependency
  - No other attribute is extraneous, so we get $F_c =$
    - `customer_id, employee_id` $\rightarrow$ `type`
    - `employee_id` $\rightarrow$ `branch_name`

3NF Decomposition : Example

- Relation schema:
  - `cust_banker_branch = (customer_id, employee_id, branch_name, type)`
- The functional dependencies for this relation schema are:
  - 1. `customer_id, employee_id` $\rightarrow$ `branch_name, type`
  - 2. `employee_id` $\rightarrow$ `branch_name`
  - 3. `customer_id, branch_name` $\rightarrow$ `employee_id`
- We first compute a canonical cover
  - `branch_name` is extraneous in the RHS of the 1st dependency
  - No other attribute is extraneous, so we get $F_c =$
    - `customer_id, employee_id` $\rightarrow$ `type`
    - `employee_id` $\rightarrow$ `branch_name`
    - `customer_id, branch_name` $\rightarrow$ `employee_id`

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`
- Observe that `(customer_id, employee_id, type)` contains a candidate key of the original schema, so no further relation schema needs be added

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`
- Observe that `(customer_id, employee_id, type)` contains a candidate key of the original schema, so no further relation schema needs be added
- At end of `for` loop, detect and delete schemas, such as `(employee_id, branch_name)`, which are subsets of other schemas

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`
- Observe that `(customer_id, employee_id, type)` contains a candidate key of the original schema, so no further relation schema needs be added
- At end of `for` loop, detect and delete schemas, such as `(employee_id, branch_name)`, which are subsets of other schemas
  - result will not depend on the order in which FDs are considered

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`
- Observe that `(customer_id, employee_id, type)` contains a candidate key of the original schema, so no further relation schema needs be added
- At end of `for` loop, detect and delete schemas, such as `(employee_id, branch_name)`, which are subsets of other schemas
  - result will not depend on the order in which FDs are considered
- The resultant simplified 3NF schema is:

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`
- Observe that `(customer_id, employee_id, type)` contains a candidate key of the original schema, so no further relation schema needs be added
- At end of `for` loop, detect and delete schemas, such as `(employee_id, branch_name)`, which are subsets of other schemas
  - result will not depend on the order in which FDs are considered
- The resultant simplified 3NF schema is:
  - `(customer_id, employee_id, type)`

3NF Decomposition : Example (Cont.)

- The `for` loop generates following 3NF schema:
  - `(customer_id, employee_id, type)`
  - `(employee_id, branch_name)`
  - `(customer_id, branch_name, employee_id)`
- Observe that `(customer_id, employee_id, type)` contains a candidate key of the original schema, so no further relation schema needs be added
- At end of `for` loop, detect and delete schemas, such as `(employee_id, branch_name)`, which are subsets of other schemas
  - result will not depend on the order in which FDs are considered
- The resultant simplified 3NF schema is:
  - `(customer_id, employee_id, type)`
  - `(customer_id, branch_name, employee_id)`

BCNF Decomposition: BCNF Definition

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form

BCNF Decomposition: BCNF Definition

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:

BCNF Decomposition: BCNF Definition

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)

BCNF Decomposition: BCNF Definition

- A relation schema R is in **BCNF** with respect to a set F of FDs if for all FDs in F^+ of the form
- $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$ at least one of the following holds:
  - $\alpha \rightarrow \beta$ is trivial (that is, $\beta \subseteq \alpha$)
  - α is a superkey for R

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$
  - Create $R_1 = AB$

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$
  - Create $R_1 = AB$
  - Create $R_2 = (R - (B - A))$

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$
  - Create $R_1 = AB$
  - Create $R_2 = (R - (B - A))$
  - **Note that**: $R_1 \cap R_2 = A$ and $A \rightarrow AB$ $(= R_1)$, so this is lossless decomposition

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$
  - Create $R_1 = AB$
  - Create $R_2 = (R - (B - A))$
  - **Note that**: $R_1 \cap R_2 = A$ and $A \rightarrow AB$ $(= R_1)$, so this is lossless decomposition
- 3. Repeat for R_1, and R_2

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$
  - Create $R_1 = AB$
  - Create $R_2 = (R - (B - A))$
  - **Note that**: $R_1 \cap R_2 = A$ and $A \rightarrow AB$ $(= R_1)$, so this is lossless decomposition
- 3. Repeat for R_1, and R_2
  - By defining F_1^+ to be all dependencies in F^+ that contain only attributes in R_1

BCNF Decomposition : Algorithm

- 1. For all dependencies $A \rightarrow B$ in F^+, check if A is a superkey
  - By using attribute closure
- 2. If not, then
  - Choose a dependency in F^+ that breaks the BCNF rules, say $A \rightarrow B$
  - Create $R_1 = AB$
  - Create $R_2 = (R - (B - A))$
  - **Note that**: $R_1 \cap R_2 = A$ and $A \rightarrow AB$ $(= R_1)$, so this is lossless decomposition
- 3. Repeat for R_1, and R_2
  - By defining F_1^+ to be all dependencies in F^+ that contain only attributes in R_1
  - Similarly F_2^+